Universidad CENFOTEC

Escuela de Ciberseguridad

Profesora: MSc. Grettel Rivera Rojas

Proyecto final

CIB-12 Aseguramiento de Redes

Elaborado por:
Génesis Castro Quesada
Jose Pablo Picado Chaves
Emmanuel Richmond Tasies

Fecha de entrega:
03 de diciembre del 2025

Desarrollo

Construcción de topología usando Virtual box

Topología usada

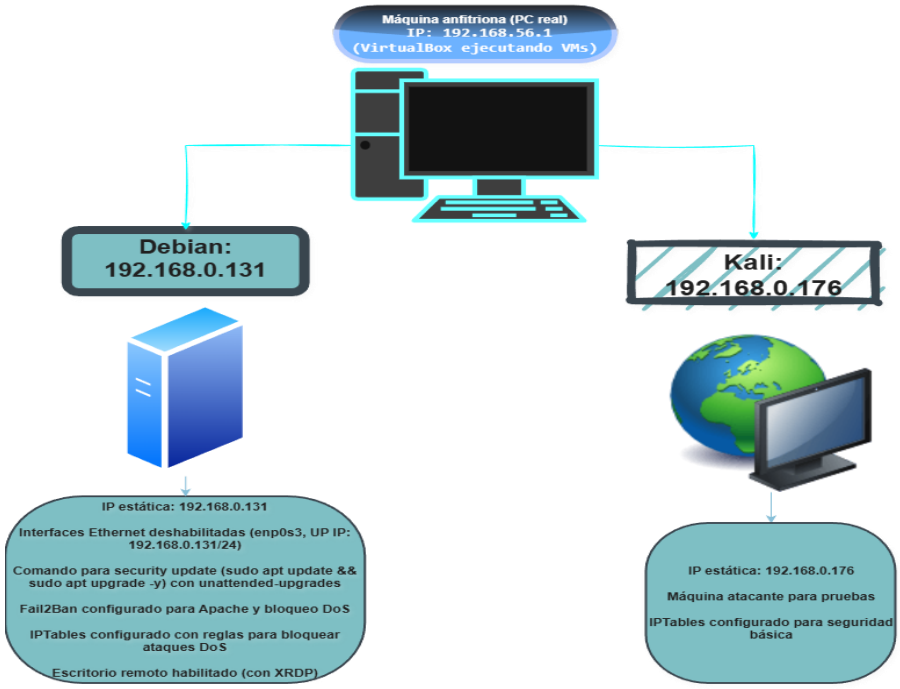


Ilustración 1 Topología usada para el proyecto

La topología del laboratorio está compuesta por una máquina anfitriona (PC real) con IP 192.168.56.1, desde la cual se ejecutan dos máquinas virtuales mediante VirtualBox: una VM Debian 12 (servidor) con IP 192.168.0.131 y una VM Kali Linux (atacante) con IP 192.168.0.176. La red utilizada es del tipo “Host-Only” para aislar el entorno de pruebas y garantizar control total sobre el tráfico generado entre las máquinas virtuales. Cada máquina virtual fue configurada según los requisitos de seguridad y pruebas definidos en el proyecto.

Configuraciones de Debian y Kali

Desactivación de las interfaces de Ethernet no requeridas. Demostración de las interfaces con su respectivo estado.

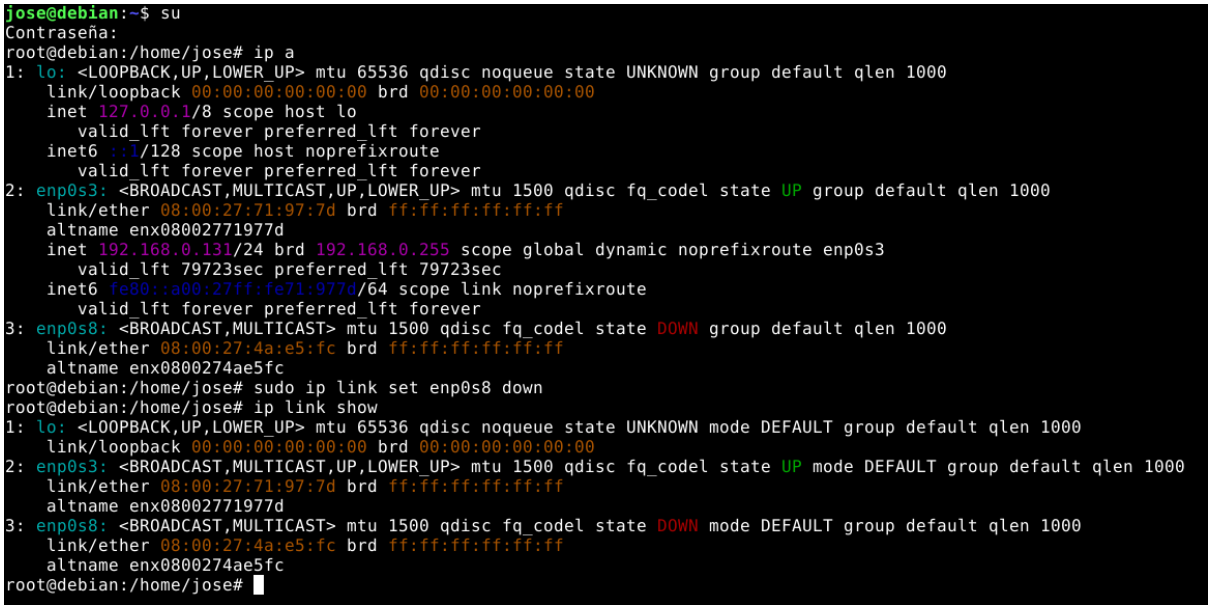


Ilustración 2 Interfaces con su respectivo estado

Se ejecutaron los comandos ip a y ip link show para visualizar todas las interfaces de red del sistema junto con su estado actual. En la salida se identifican las siguientes interfaces:

- **lo (Loopback):** interfaz interna del sistema, activa (UP), utilizada para la comunicación local con la dirección IP 127.0.0.1.
- **enp0s3:** interfaz de red principal con dirección IP 192.168.0.131, activa (UP) y conectada a la red local.
- **enp0s8:** interfaz secundaria, inicialmente mostraba el estado DOWN. Se ejecutó el comando:
  sudo ip link set enp0s8 down

La ejecución del comando se realizó para asegurar la desactivación completamente, ya que no es requerida.

Finalmente, se verificó nuevamente con **ip link show** que **enp0s8** permaneciera en estado **DOWN**, confirmando que fue deshabilitada correctamente y que solo las interfaces necesarias permanecen activas.

IP estática para las interfaces que se requieren utilizar

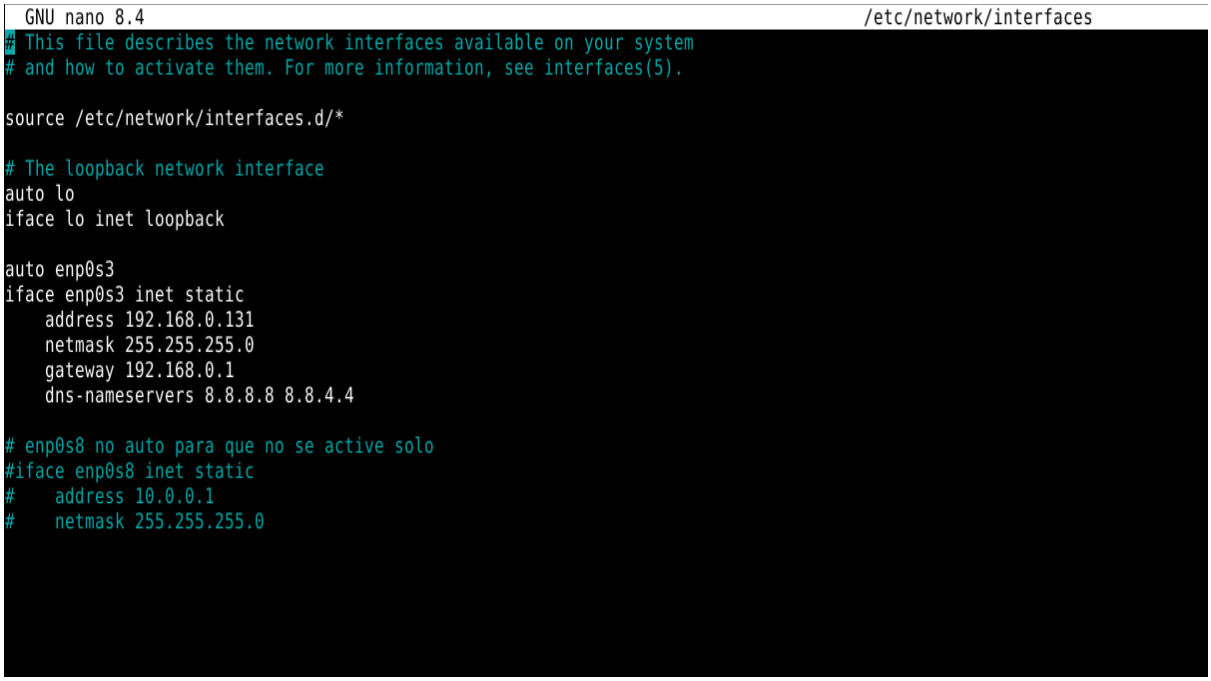


Ilustración 3 Configuración dentro de /etc/network/interfaces

Para configurar una IP estática, nos dirigimos al archivo de configuración de red ubicado en **/etc/network/interfaces**. En este archivo, detectamos que la interfaz de red principal es **enp0s3**, que tiene asignada la dirección IP estática **192.168.0.131**. Esta interfaz se encuentra activa (UP) y conectada a la red local, por lo tanto, es la que utilizaremos para la comunicación principal del sistema. Además, configuramos la interfaz enp0s8 para que no se active automáticamente, evitando que interfiera en la red mientras no sea necesaria. Esta gestión de interfaces permite un control más preciso del tráfico y asegura que solo las interfaces necesarias estén operativas con IPs asignadas, fortaleciendo así la estabilidad y seguridad de la configuración de red.

Security update de las librerías instaladas

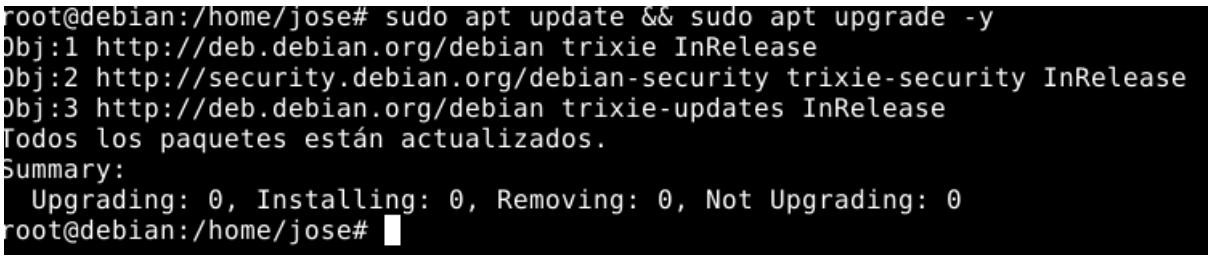


Ilustración 4 Security Update de las librerías utilizadas

Para mantener el sistema actualizado y seguro, realizamos la actualización de las librerías instaladas mediante el comando sudo **apt update && sudo apt upgrade -y.** Este comando actualiza la lista de paquetes disponibles y aplica las actualizaciones automáticamente sin requerir intervención durante el proceso.

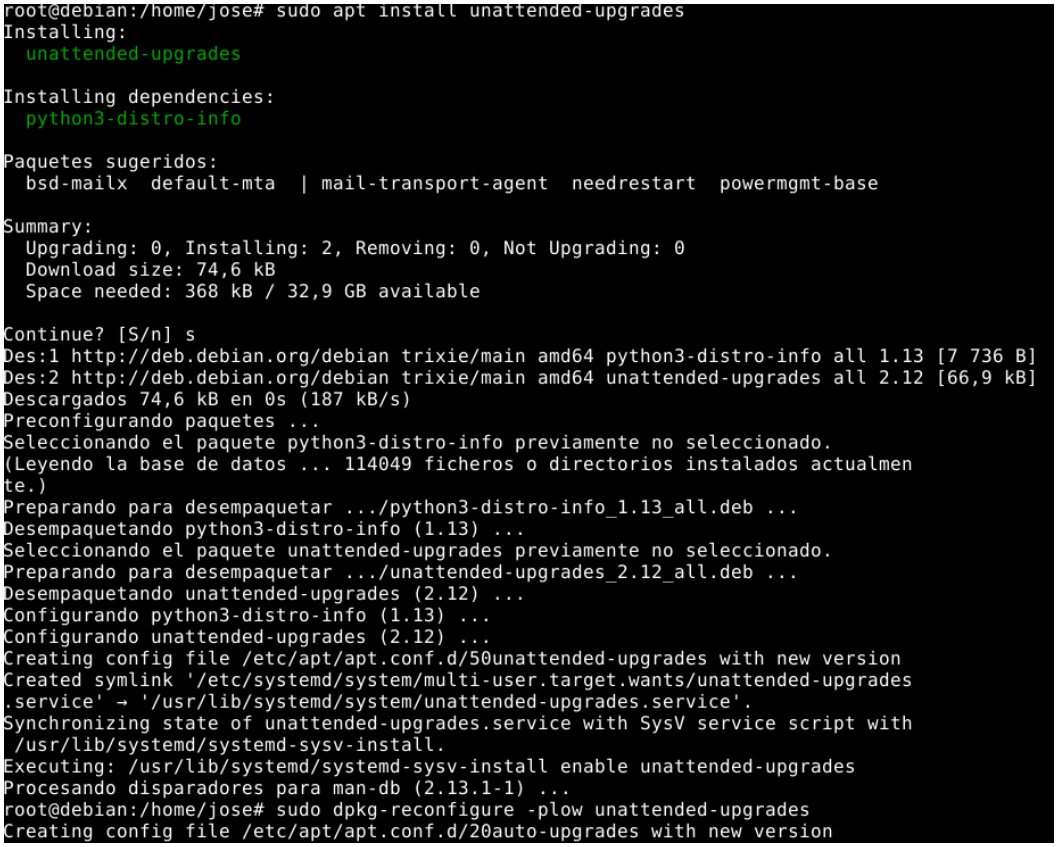


Ilustración 5 Unattended-upgrades

Como complemento para mejorar la seguridad y evitar que el sistema quede sin actualizar, se configuró el paquete **unattended-upgrades**. Este servicio permite que las actualizaciones de seguridad se realicen de forma automática en segundo plano, descargando e instalando parches críticos sin necesidad de comando manual, lo que reduce riesgos por vulnerabilidades sin parchear.

Fail2Ban para filtrar bad bods en el servidor Apache

```
root@debian:/home/jose# sudo apt install fail2ban -y
Installing:
  fail2ban

Installing dependencies:
  python3-autocommand       python3-more-itertools  python3-typing-extensions
  python3-inflect           python3-pkg-resources   python3-zipp
  python3-jaraco.context    python3-setuptools      whois
  python3-jaraco.functools  python3-systemd
  python3-jaraco.text       python3-typeguard

Paquetes sugeridos:
  mailx  system-log-daemon  monit  sqlite3  python-setuptools-doc

Summary:
  Upgrading: 0, Installing: 14, Removing: 0, Not Upgrading: 0
  Download size: 1 826 kB
  Space needed: 8 951 kB / 32,9 GB available

Des:1 http://deb.debian.org/debian trixie/main amd64 python3-systemd amd64 235-1+b6 [40,8 kB]
Des:2 http://deb.debian.org/debian trixie/main amd64 fail2ban all 1.1.0-8 [466 kB]
Des:3 http://deb.debian.org/debian trixie/main amd64 python3-autocommand all 2.2.2-3 [13,6 kB]
Des:4 http://deb.debian.org/debian trixie/main amd64 python3-more-itertools all 10.7.0-1 [67,4 kB]
Des:5 http://deb.debian.org/debian trixie/main amd64 python3-typing-extensions all 4.13.2-1 [90,5 kB]
Des:6 http://deb.debian.org/debian trixie/main amd64 python3-typeguard all 4.4.2-1 [37,3 kB]
Des:7 http://deb.debian.org/debian trixie/main amd64 python3-inflect all 7.3.1-2 [32,4 kB]
Des:8 http://deb.debian.org/debian trixie/main amd64 python3-jaraco.functools all 4.1.0-1 [12,0 kB]
Des:9 http://deb.debian.org/debian trixie/main amd64 python3-pkg-resources all 78.1.1-0.1 [224 kB]
Des:10 http://deb.debian.org/debian trixie/main amd64 python3-jaraco.text all 4.0.0-1 [11,4 kB]
Des:11 http://deb.debian.org/debian trixie/main amd64 python3-zipp all 3.21.0-1 [10,6 kB]
Des:12 http://deb.debian.org/debian trixie/main amd64 python3-setuptools all 78.1.1-0.1 [738 kB]
Des:13 http://deb.debian.org/debian trixie/main amd64 python3-jaraco.context all 6.0.1-1 [8 276 B]
Des:14 http://deb.debian.org/debian trixie/main amd64 whois amd64 5.6.3 [72,4 kB]
Descargados 1 826 kB en 1s (1 492 kB/s)
```

Ilustración 6 Instalación de fail2ban

Al iniciar se debe instalar Fail2ban en el sistema para protegerlo contra ataques de fuerza bruta y accesos no autorizados. Fail2ban es una herramienta que monitorea los intentos de login fallidos y bloquea automáticamente las IPs que hacen intentos repetidos, mediante reglas de firewall.

```
[apache-badbots]
enabled = true
port = http,https
filter = apache-badbots
logpath = /var/log/apache2/*access.log
maxretry = 1
findtime = 86400
bantime = 259200
action = iptables[name=BadBots, port=http, protocol=tcp]
```

Ilustración 7 Bloque para jail.local contra bad bots

Después de instalar Fail2Ban, debes editar el archivo de configuración de prisiones ubicado en **nano /etc/fail2ban/jail.local.** Esta configuración indica a Fail2Ban que monitoree los logs de acceso de Apache, usando el filtro **apache-badbots** para detectar patrones de bots maliciosos. Cuando una IP coincida con el filtro una sola vez (maxretry

1) dentro de 24 horas (findtime 86400 segundos), será bloqueada durante 3 días (bantime 259200 segundos) mediante reglas de iptables. Es importante reiniciar el servicio Fail2Ban luego de hacer cambios usando **sudo systemctl restart fail2ban** para que la configuración surta efecto.

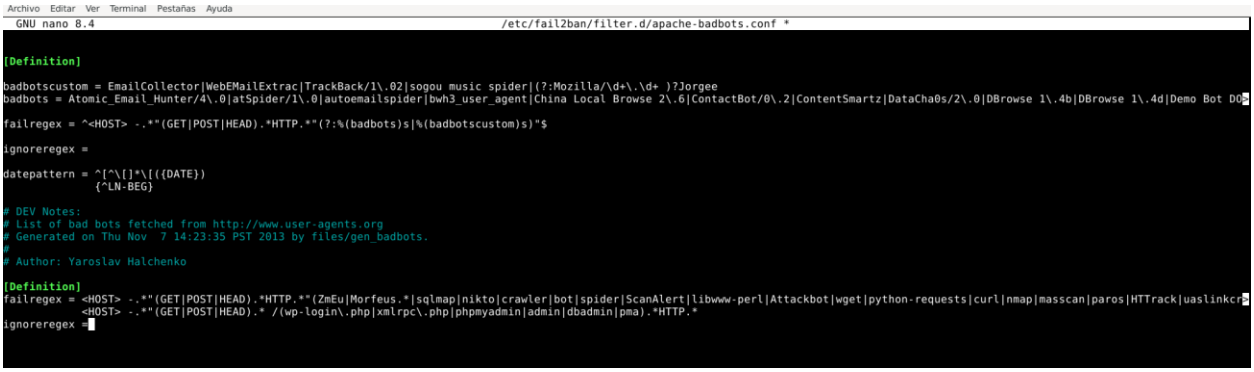


Ilustración 8 nano filter.d/ apache-badbots.conf

Este archivo define el filtro que Fail2Ban empleará para reconocer bots maliciosos y accesos sospechosos a través de expresiones regulares que analizan las entradas de los logs de acceso de Apache. Busca User-Agents y patrones comunes de bots conocidos y reglas que detectan accesos a rutas sensibles como **/wp-login.php, /xmlrpc.php, /phpmyadmin,** entre otros.

Cuando una IP coincide con alguno de estos patrones, Fail2Ban la marca para ser bloqueada según lo definido en la cárcel que use este filtro. Este filtro ayuda a prevenir que bots automatizados exploren o ataquen el servidor, contribuyendo a mejorar la seguridad y estabilidad del mismo.



Ilustración 9 sudo fail2ban-client status apache

Después de configurar el filtro en **/etc/fail2ban/filter.d/apache-badbots.conf**, es fundamental verificar el estado de Fail2Ban para asegurarse de que está funcionando correctamente. Fail2Ban debe estar monitoreando los logs de Apache y bloqueando efectivamente las direcciones IP sospechosas que intentan generar tráfico malicioso o bots no deseados.

Para ello, se utiliza el comando:

sudo fail2ban-client status apache-badbots

Este comando muestra información detallada sobre la cárcel (jail) llamada "apache-badbots", incluyendo:

- La cantidad de intentos fallidos detectados que han activado la regla.
- Las direcciones IP actualmente bloqueadas por el filtro.
- El archivo de log que Fail2Ban está monitoreando, que generalmente es **/var/log/apache2/error.log** o el equivalente en tu sistema.

Es importante revisar periódicamente esta información para entender qué tipo de actividad está siendo bloqueada y ajustar los filtros si es necesario para evitar falsos positivos o permitir acciones legítimas

```
root@debian:/home/jose# sudo apt install iptables
Installing:
  iptables

Installing dependencies:
  libip4tc2  libip6tc2

Paquetes sugeridos:
  firewalld

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 401 kB
  Space needed: 2 599 kB / 32,8 GB available
```

Ilustración 10 Instalación de iptables

Primero se debe instalar iptables en Debian, primero se debe actualizar la lista de paquetes con **sudo apt update**. Luego, se instala iptables con el comando **sudo apt install iptables.**

```
root@debian:/home/jose# sudo apt install iptables-persistent
iptables-persistent ya está en su versión más reciente (1.0.23).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
root@debian:/home/jose#
```

Ilustración 11 Instalación de paquete iptables-persistent

Además, es importante instalar también el paquete **iptables-persistent** para que las reglas configuradas en iptables se mantengan persistentes después de reiniciar el sistema. Esto se hace con **sudo apt install iptables-persistent**, durante la instalación de **iptables-persistent**, se solicitará guardar las reglas actuales, lo cual permite que se carguen automáticamente en cada arranque mediante archivos en **/etc/iptables/rules.v4** para IPv4 y **/etc/iptables/rules.v6** para IPv6. Así, cualquier configuración de firewall que hayas realizado no se perderá tras reiniciar el servidor.

```
root@debian:/home/jose# sudo systemctl enable netfilter-persistent
Synchronizing state of netfilter-persistent.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable netfilter-persistent
root@debian:/home/jose# sudo systemctl start netfilter-persistent
root@debian:/home/jose# sudo iptables -A INPUT -m state --state INVALID -j DROP
root@debian:/home/jose# sudo iptables -A INPUT -p tcp ! --syn -m state --state NEW -j DROP
root@debian:/home/jose# sudo iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 20 -j REJECT
root@debian:/home/jose# sudo iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --update --seconds 60 --hitcount 30 -j DROP
root@debian:/home/jose# sudo iptables -N syn_flood
root@debian:/home/jose# sudo iptables -A INPUT -p tcp --syn --dport 80 -m limit --limit 10/s --limit-burst 20 -j RETURN
root@debian:/home/jose# sudo iptables -A INPUT -p tcp --syn --dport 80 -j DROP
root@debian:/home/jose#
```

Ilustración 12 Reglas para bloquear ataques DoS HTTP

iptables -A INPUT -m state --state INVALID -j DROP

- Los paquetes marcados como "INVALID" por el sistema de seguimiento de conexiones suelen indicar intentos de manipulación o errores, típicos en ataques DoS y escaneos agresivos.

1. iptables -A INPUT -p tcp ! --syn -m state --state NEW -j DROP
   - Impide paquetes que intenten iniciar conexiones sin tener el flag SYN (sincronización), evitando sesiones maliciosas y exploits que no respetan el protocolo estándar.

2. iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 20 -j REJECT
   - Previene que una sola IP haga más de 20 conexiones simultáneas al puerto HTTP, mitigando ataques de saturación masiva.

3. iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --update --seconds 60 --hitcount 30 -j DROP
   - Si una IP inicia más de 30 conexiones nuevas en un minuto será bloqueada, limitando el efecto de flooding de conexiones rápidas.

4. iptables -N syn_flood
5. iptables -A INPUT -p tcp --syn --dport 80 -m limit --limit 10/s --limit-burst 20 -j

RETURN

6. iptables -A INPUT -p tcp --syn --dport 80 -j DROP

Estos límites permiten controlar el número de paquetes SYN (sincronización) por segundo, reduciendo el impacto de ataques SYN flood que intentan colapsar el servidor.

```
[apache-get-dos]
enabled = true
port = http,https
filter = apache-get-dos
logpath = /var/log/apache2/access.log
maxretry = 120
findtime = 60
bantime = 3600
action = iptables[name=apache-get-dos, port=http, protocol=tcp]
```

Ilustración 13 Fail2ban. Prisión para ataques DoS HTTP

Datos por considerar:

- enabled = true

Activa la cárcel para que Fail2Ban la use automáticamente.

- port = http,https

Asegura la cobertura tanto para HTTP (80) como HTTPS (443).

- filter = apache-get-dos

Permite definir un filtro personalizado en /etc/fail2ban/filter.d/apache-get-dos.conf que detecte patrones de solicitudes sospechosas (por ejemplo, muchos GET seguidos o respuestas con error 4xx/5xx).

- logpath = /var/log/apache2/access.log

Analiza todos los accesos a Apache en busca de actividad anormal, fundamental para detectar DoS genuinos.

- maxretry = 120

Reduce el umbral a 120 peticiones en el intervalo definido, permitiendo bloquear ataques medianamente intensos sin afectar a usuarios legítimos.

- findtime = 60

Si una IP supera el número permitido en 60 segundos, se considera sospechosa y se bloquea.

- bantime = 3600

Aumenta el bloqueo a 1 hora, fortaleciendo la mitigación al dar tiempo al administrador para responder y analizar si el ataque continúa.

- action = iptables[name=apache-get-dos, port=http, protocol=tcp]

Aplica el bloqueo a nivel de firewall, asegurando que la IP ofensiva ya no pueda atacar al servidor durante el tiempo de sanción.



Ilustración 14 Filtro para /etc/fail2ban/filter.d/apache-get-dos.confi

Puntos por considerar:

- Primera línea: Detecta cualquier petición GET, POST o HEAD al servidor Apache, cubriendo el tipo más común de ataques volumen y flooding.

- Segunda línea: Filtra intentos de acceso a rutas sensibles típicamente objetivo de bots y ataques DoS (como wp-login, xmlrpc, panel de administración, phpmyadmin).

- Tercera línea: Detecta intentos de acceso a archivos ocultos y de configuración (por ejemplo. env, .git, .htaccess), que suelen ser explotados durante escaneos y ataques automatizados.

- Cuarta línea: Captura tráfico anómalo que provoca respuestas de error del servidor (códigos HTTP 404, 403, 500, 502, 503, 504), frecuente en ataques de exploración y saturación.

¿Cómo funciona la combinación?

- IPTables bloquea el tráfico masivo y anómalo antes de llegar a Apache.
- Fail2Ban observa el comportamiento en los logs del servidor web y bloquea IPs que rompen los límites preestablecidos, incluso para ataques más lentos y sofisticados.
- La unión mejora la protección tanto a nivel de red como de aplicación.

IP estática para atacante

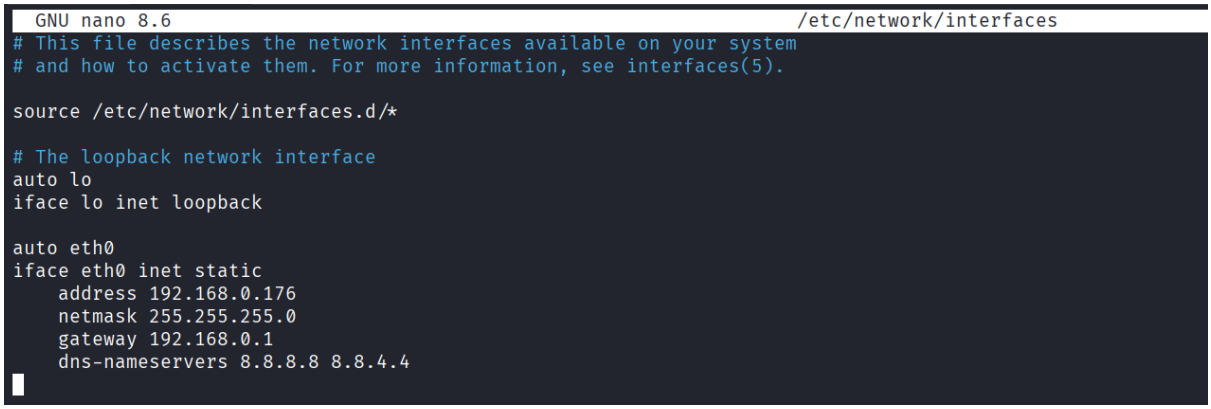


Ilustración 15 Configuración dentro de /etc/network/intercases en Kali

En esta sección se configuró la interfaz de red eth0 con una dirección IP estática para asegurar que el equipo mantenga siempre la misma identidad en la red local.

- **auto eth0** indica que la interfaz debe activarse automáticamente al iniciar el sistema.

- **iface eth0 inet static** define que la configuración de red será estática, es decir, sin usar DHCP.
- **address 192.168.0.176** es la dirección IP fija asignada a eth0.
- **netmask 255.255.255.0** especifica la máscara de subred para delimitar la red local.
- **gateway 192.168.0.1** designa la puerta de enlace predeterminada para salir a otras redes o internet.
- **dns-nameservers 8.8.8.8 8.8.4.4** fijan los servidores DNS públicos de Google para la resolución de nombres de dominio.

IPtables configuración básica

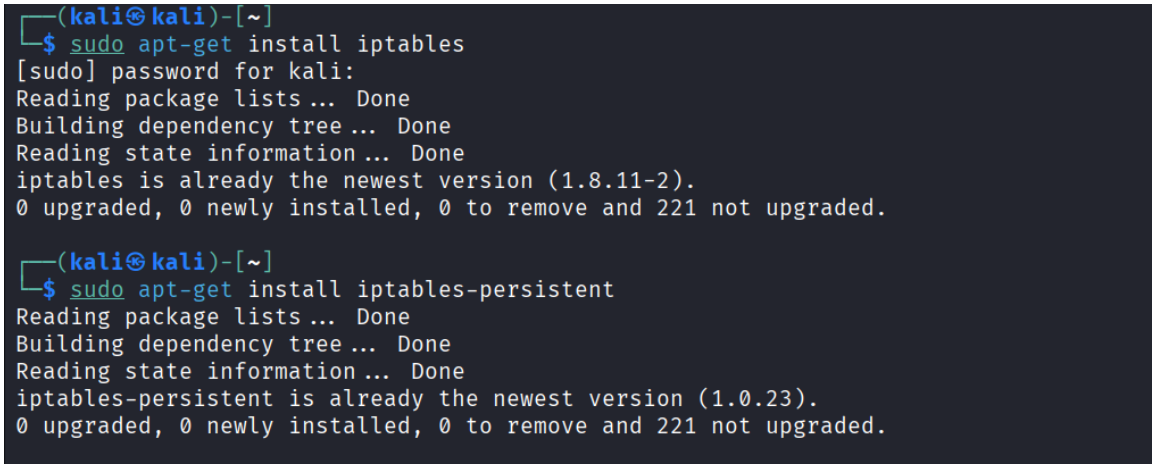


Ilustración 16 Instalación de iptables en Kali

Primero se intala **iptables y iptables-persistent** para poder gestionar las reglas del firewall y asegurar que se mantengan tras reiniciar el sistema. Luego se configuran reglas básicas para permitir tráfico necesario y proteger el equipo. Finalmente, se guardan las reglas para que sean permanentes y el firewall funcione correctamente siempre.

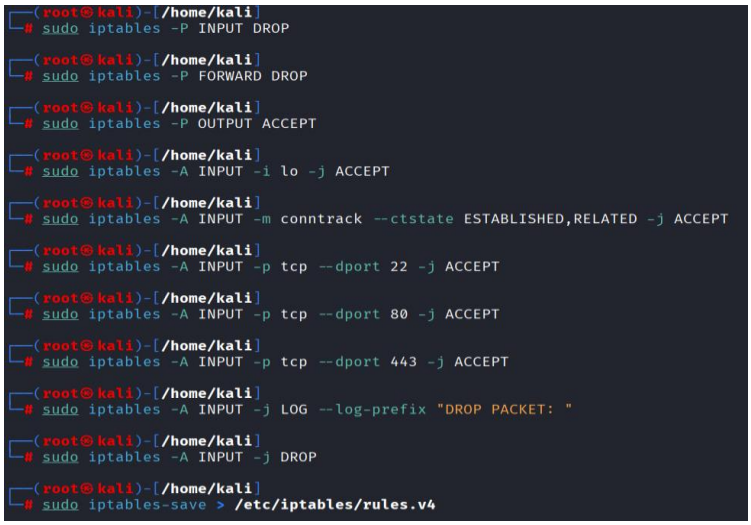


Ilustración 17 Configuración básica de iptables en Kali Linux

Aspectos por considerar:

1. **sudo iptables -P INPUT DROP**
    - Limpia todas las reglas actuales para empezar desde cero.
2. **sudo iptables -P FORWARD DROP**
    - Establece que por defecto se bloquee todo el tráfico entrante.
3. **sudo iptables -P OUTPUT ACCEPT**
    - Permite todo el tráfico saliente para que el sistema pueda enviar datos sin restricciones.
4. **sudo iptables -A INPUT -i lo -j ACCEPT**
    - Permite todo el tráfico en la interfaz local de loopback (comunicaciones internas del sistema).
5. **sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT**
    - Permite paquetes entrantes que son respuestas o relacionados a conexiones ya establecidas por el sistema (importante para respuestas de consultas salientes).
6. **sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT**
    - Permite conexiones entrantes TCP al puerto 22 (SSH).
7. **sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT**
    - Permite tráfico HTTP.

8. **sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT**
    - Permite tráfico HTTPS.
9. **sudo iptables -A INPUT -j LOG --log-prefix "DROP PACKET "**
    - Registra en el log los paquetes que no coinciden con reglas anteriores y serán bloqueados.
10. **sudo iptables -A INPUT -j DROP**
    - Finalmente, descarta todos los paquetes que no hayan sido permitidos explícitamente.

Escritorio remoto para la máquina

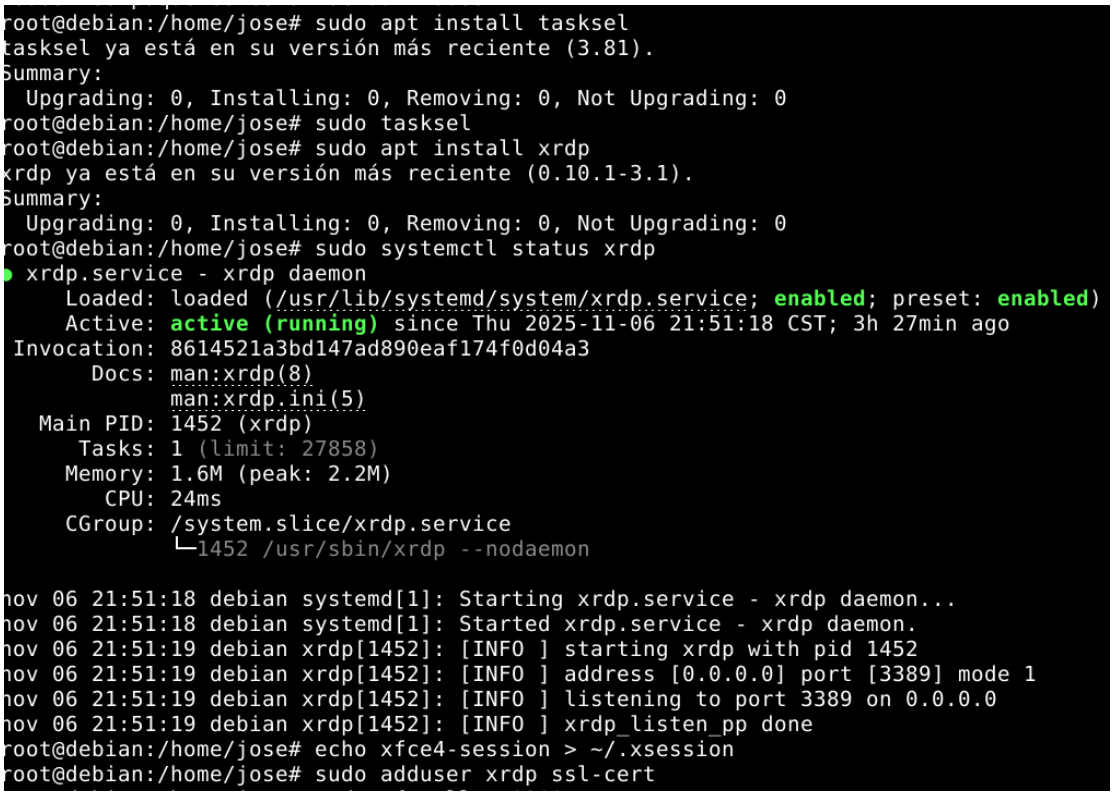


Ilustración 18 Instalación de XRDP y configuración de la sesión de escritorio

1. Instalar XRDP

Una vez que tu servidor Debian tiene un entorno de escritorio, el siguiente paso es instalar el servidor XRDP con **sudo apt install xrdp** y verificar que el servicio esté activo con **sudo systemctl status xrdp**. XRDP es un servicio que permite ofrecer sesiones gráficas remotas a través del protocolo RDP, que es ampliamente utilizado, sobre todo

en sistemas Windows, para conexiones de escritorio remoto. La gran ventaja de XRDP es que es compatible con clientes estándar (como el que trae Windows por defecto, y muchos clientes para Linux y macOS), por lo que no necesitas software adicional para conectarte remotamente a tu Debian. Solo necesitas saber la dirección IP del servidor y tus credenciales. La instalación de XRDP es sencilla y se realiza normalmente con el paquete oficial disponible en los repositorios de Debian. Al instalarlo, automáticamente queda configurado para ejecutarse como servicio y aceptar conexiones entrantes desde el puerto estándar 3389.

2. Configurar la sesión gráfica para XRDP

XRDP, cuando recibe una conexión remota, necesita saber qué entorno gráfico debe iniciar para presentar al usuario. Aquí es donde se establece la relación entre XRDP y XFCE. Para configurarlo, es necesario indicar explícitamente que cuando alguien inicie sesión remota, se le debe cargar una sesión de XFCE por eso el **echo xfce4-session > ~/.xsession**. Una forma común y efectiva es crear un archivo oculto .**xsession** en el directorio home del usuario, con el contenido **xfce4-session**. Esto le dice a XRDP que inicie esa sesión de escritorio. Además, para garantizar que el servicio XRDP funcione sin problemas de permisos, se agrega el usuario **xrdp** al grupo **ssl-cert** (**sudo adduser xrdp ssl-cert**). Esto es importante porque XRDP utiliza certificados SSL para las conexiones seguras, y sin permisos adecuados podría fallar la sesión gráfica.

Esta configuración garantiza que al conectar por RDP, el usuario tenga el entorno XMCE cargado correctamente, evitando errores comunes como pantalla negra y asegurando una sesión gráfica funcional y estable.

Implementación de DVWA en Debian con Apache, MariaDB y PHP

Instalación de Apache, MariaDB y PHP en Debian

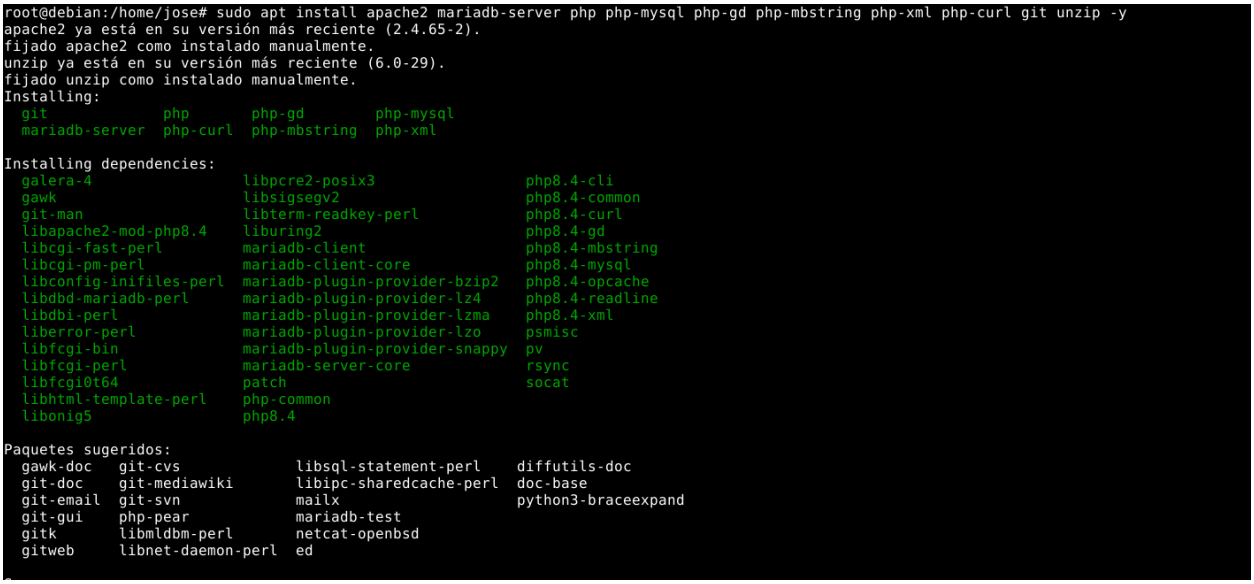


Ilustración 19 Comandos utilizados para instalar Apache, MariaDB y PHP

En este paso se ejecuta el comando **sudo apt install apache2 mariadb-server php php-mysql php-gd php-mbstring php-xml php-curl git unzip -y**, el cual utiliza el gestor de paquetes APT con privilegios administrativos para instalar todos los componentes necesarios que nos van a permitir usar **DVWA** (**Damn Vulnerable Web Application**) en Debian. El parámetro -y confirma automáticamente la instalación sin requerir intervención del usuario. Durante este proceso se instalan los paquetes principales como **apache2** (servidor web encargado de alojar y servir la aplicación), **mariadb-server** (sistema de gestión de bases de datos donde se almacenará la información del sitio) y **php** (lenguaje de programación interpretado del lado del servidor que procesa el código de DVWA). Además, se añaden módulos adicionales de PHP necesarios para su correcto funcionamiento: **php-mysql** (permite la conexión entre PHP y MariaDB), **php-gd** (habilita el procesamiento de imágenes), **php-mbstring** (proporciona compatibilidad con cadenas multibyte), **php-xml** (permite trabajar con archivos XML) y **php-curl** (facilita la transferencia de datos entre servidores). También se incluyen herramientas de apoyo como **git** (para clonar el repositorio de DVWA) y **unzip** (para descomprimir archivos ZIP). Durante la instalación, el sistema indica qué paquetes ya se encuentran actualizados y muestra las dependencias adicionales que serán instaladas para garantizar el correcto funcionamiento del entorno en Debian.

Inicio y habilitación de los servicios Apache y Maria DB

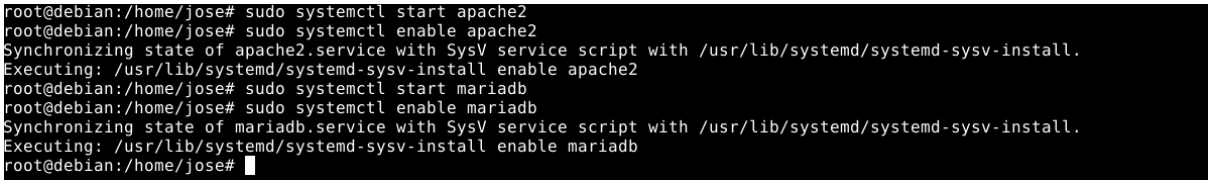


Ilustración 20 Ejecución de comandos para iniciar y habilitar los servicios

Se ejecutaron los comandos **sudo systemctl start apache2** y sudo **systemctl start mariadb** para iniciar los servicios de Apache y MariaDB, respectivamente, permitiendo que ambos comiencen a ejecutarse y estén disponibles para el funcionamiento de la aplicación **DVWA (Damn Vulnerable Web Application)**. Posteriormente, se utilizan los comandos **sudo systemctl enable apache2 y sudo systemctl enable mariadb** para habilitar los servicios, lo que garantiza que se inicien automáticamente cada vez que el sistema arranque. Este proceso sincroniza el estado de los servicios con **systemd**, asegurando que tanto el servidor web Apache como el gestor de base de datos MariaDB se activen de forma permanente y automática, proporcionando así la base necesaria para ejecutar y administrar DVWA correctamente.

Seguridad básica y configuración inicial de MariaDB



Ilustración 21 Aplicación de comando para asegurar el servicio Mariadb

Después de instalar MariaDB en Debian, es fundamental asegurar la base de datos para evitar accesos no autorizados y reducir riesgos de seguridad comunes en instalaciones por defecto. Para esto, se ejecuta la herramienta interactiva **mariadb-secure-installation**, que guía paso a paso por configuraciones básicas pero esenciales. En este proceso, se establece la contraseña para el usuario administrativo root de MariaDB (en este caso, la contraseña elegida fue "Cenfo2025"). Además, el script pregunta si se desean eliminar usuarios anónimos que podrían dar acceso sin autenticar, deshabilitar el acceso remoto para root para restringir conexiones solo desde la máquina local, y eliminar bases de datos de pruebas que no deberían quedar en un entorno productivo.

Descarga e instalación de DVWA en el directorio raíz web y configuración de permisos adecuados



Ilustración 22 sudo git clone https://github.com/digininja/DVWA.git dvwa

Para instalar DVWA (Damn Vulnerable Web Application) en un entorno Debian, es importante ubicar los archivos en el directorio raíz del servidor web, típicamente **/var/www/html**. Usando el comando git clone, se descarga el código fuente oficial de DVWA desde GitHub directamente a este directorio. Después, se ajusta la propiedad de los archivos y carpetas para que el usuario y grupo del servidor web (**www-data**) pueda acceder y manipularlos sin restricciones.

Por último, se aplican permisos de acceso adecuados con **chmod -R 755**, que permiten la lectura y ejecución de los archivos, asegurando que Apache pueda servirlos correctamente y que los usuarios puedan interactuar con la aplicación vulnerable desde el navegador. Este procedimiento es esencial para garantizar que DVWA esté disponible y funcional para prácticas de seguridad, pruebas de penetración y análisis en entornos controlados, manteniendo la integridad de los archivos y la correcta interacción entre el

servidor web y la aplicación.

Creación de base de datos y usuario para DVWA en MariaDB

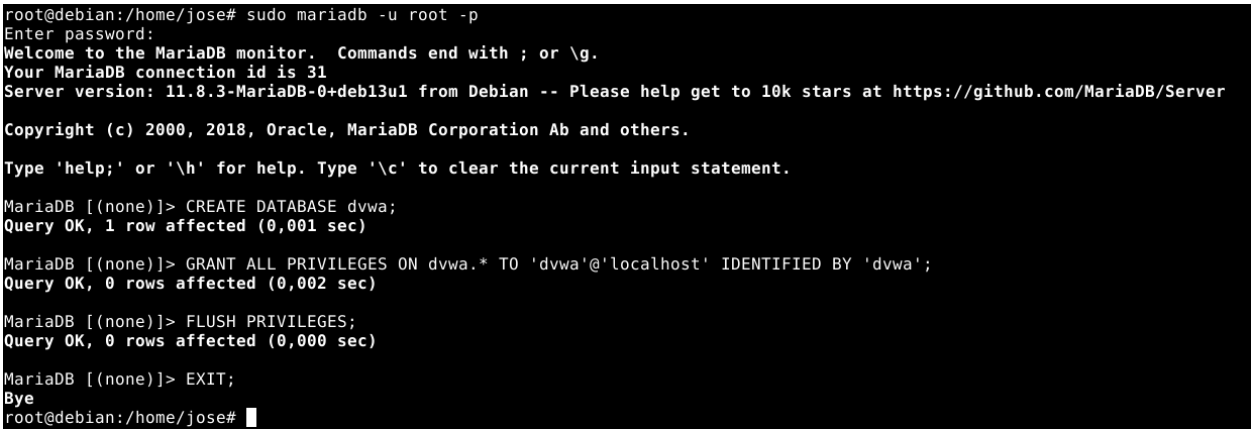


Ilustración 23 Preparación de base de datos

Después de instalar DVWA y ajustar los permisos en el directorio web, el siguiente paso esencial es preparar la base de datos que la aplicación utilizará. Esto se realiza accediendo a MariaDB con el usuario root (por ejemplo, usando **sudo mariadb -u root -p**) y ejecutando una serie de comandos para crear la base de datos exclusiva para DVWA, un usuario dedicado y otorgarle los privilegios necesarios.

Los comandos típicos son:

CREATE DATABASE dvwa;

CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';

GRANT ALL ON dvwa.* TO 'dvwa'@'localhost';

FLUSH PRIVILEGES;

- **CREATE DATABASE dvwa;** Crea una base de datos específica para la aplicación DVWA, evitando mezclas de datos y facilitando la administración.
- **CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';** Añade un usuario limitado solo para la aplicación, con su propia contraseña segura.
- ***GRANT ALL ON dvwa. TO 'dvwa'@'localhost';**** Otorga todos los privilegios necesarios para operar la base de datos DVWA, pero solo sobre esa base de datos.

- **FLUSH PRIVILEGES;** Recarga la configuración de usuarios y privilegios para aplicar los cambios de inmediato.

Esta separación entre bases de datos y usuarios asegura que DVWA funcione correctamente y facilita la administración segura.

Configuración de conexión a base de datos en DVWA

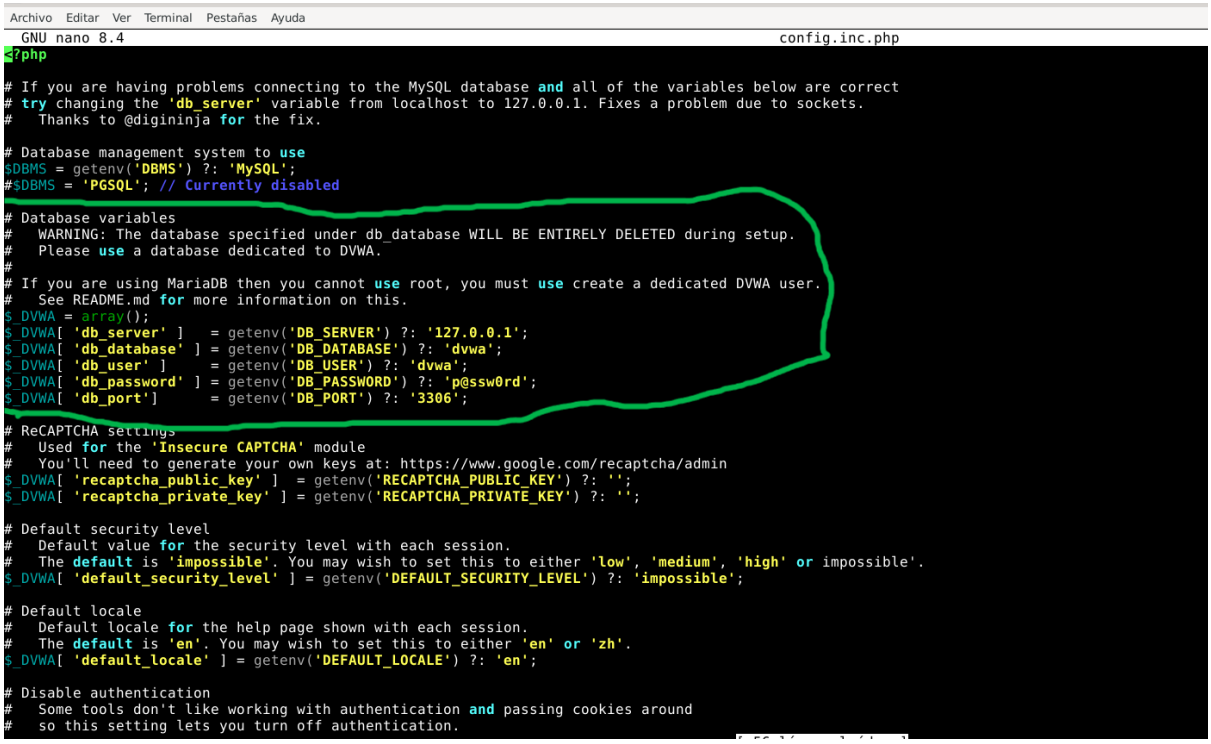


Ilustración 24 Configuración de la aplicación

Después de crear la base de datos y el usuario para DVWA en MariaDB, es necesario configurar la aplicación para que utilice estos datos de acceso correctamente. El procedimiento consiste en moverse al directorio de configuración de DVWA (**cd /var/www/html/dvwa/config**), luego realizar una copia del archivo de ejemplo de configuración con **sudo cp config.inc.php.dist config.inc.php**. A continuación, edita el archivo **config.inc.php** con un editor como nano (**sudo nano config.inc.php**). Dentro, localiza la sección "Database variables" y modifica las siguientes líneas para que coincidan con los valores de la base de datos creada previamente:

$_DVWA['db_server'] = '127.0.0.1';

$_DVWA['db_database'] = 'dvwa';

$_DVWA['db_user'] = 'dvwa';

$_DVWA['db_password'] = 'p@ssw0rd';

$_DVWA['db_port'] = '3306';

Este ajuste asegura que DVWA pueda conectarse correctamente a MariaDB utilizando el usuario dedicado **dvwa**.

Creación y Configuración de la Base de Datos en DVWA

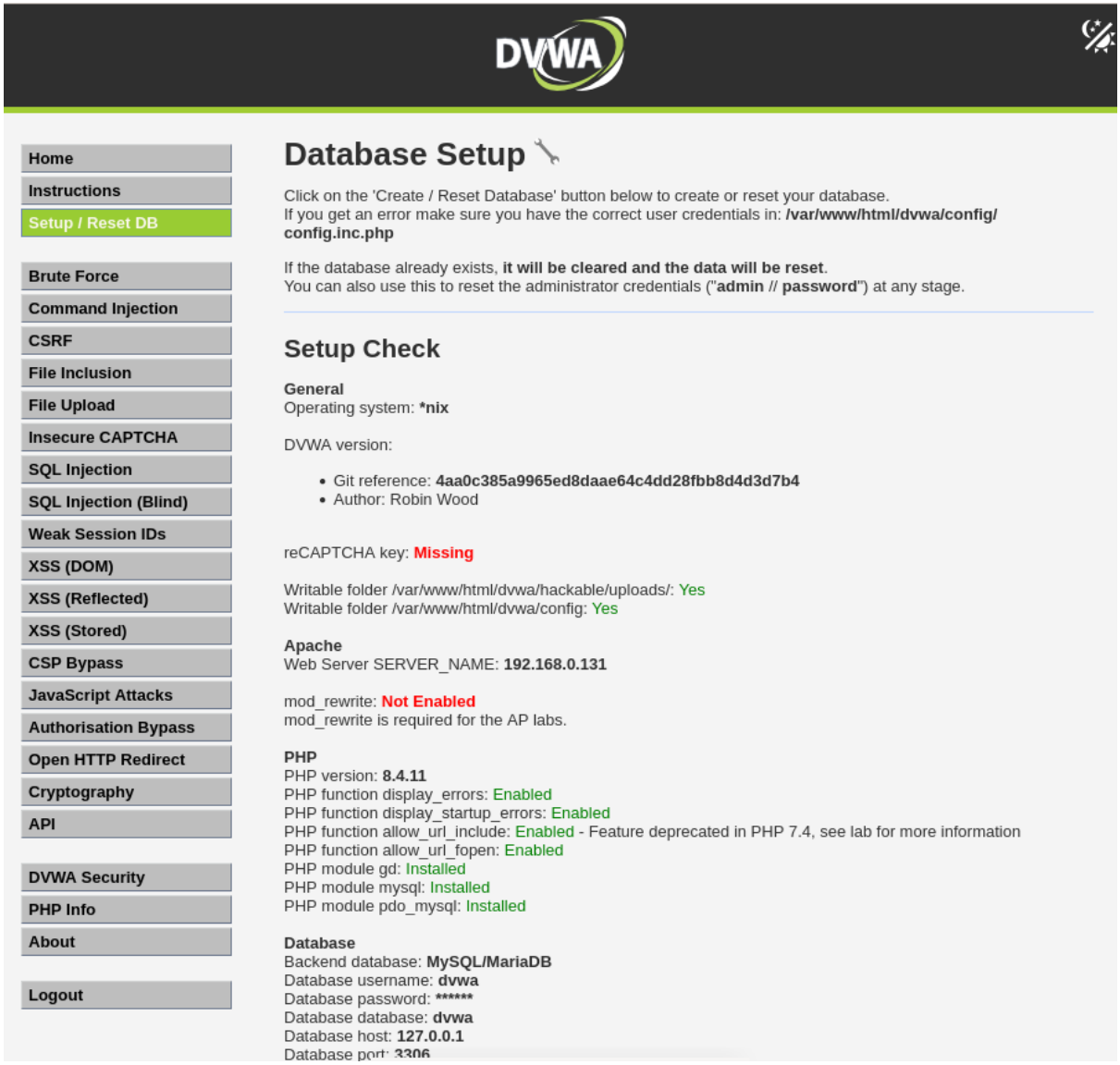


Ilustración 25 Acceso a DVWA desde el navegador

En el navegador de la máquina Debian se abre la dirección **http://192.168.0.131/dvwa**.

Desde la interfaz web se ingresa a la sección **Setup / Reset DB**, donde DVWA verifica los requisitos del entorno como la versión de PHP, permisos de carpetas (**config** y **uploads**), módulos instalados y conexión con la base de datos. Una vez revisado todo, hacemos clic en el botón **Create / Reset Database**, lo que permite a DVWA conectarse con MySQL/MariaDB y crear automáticamente las tablas necesarias junto con los datos iniciales, incluyendo el usuario administrador por defecto (**admin / password**). Si el proceso es exitoso, se mostrará un mensaje confirmando que la base de datos se creó correctamente y el sistema quedará listo para iniciar sesión.

Inicio de sesión exitoso en DVWA — admin / password

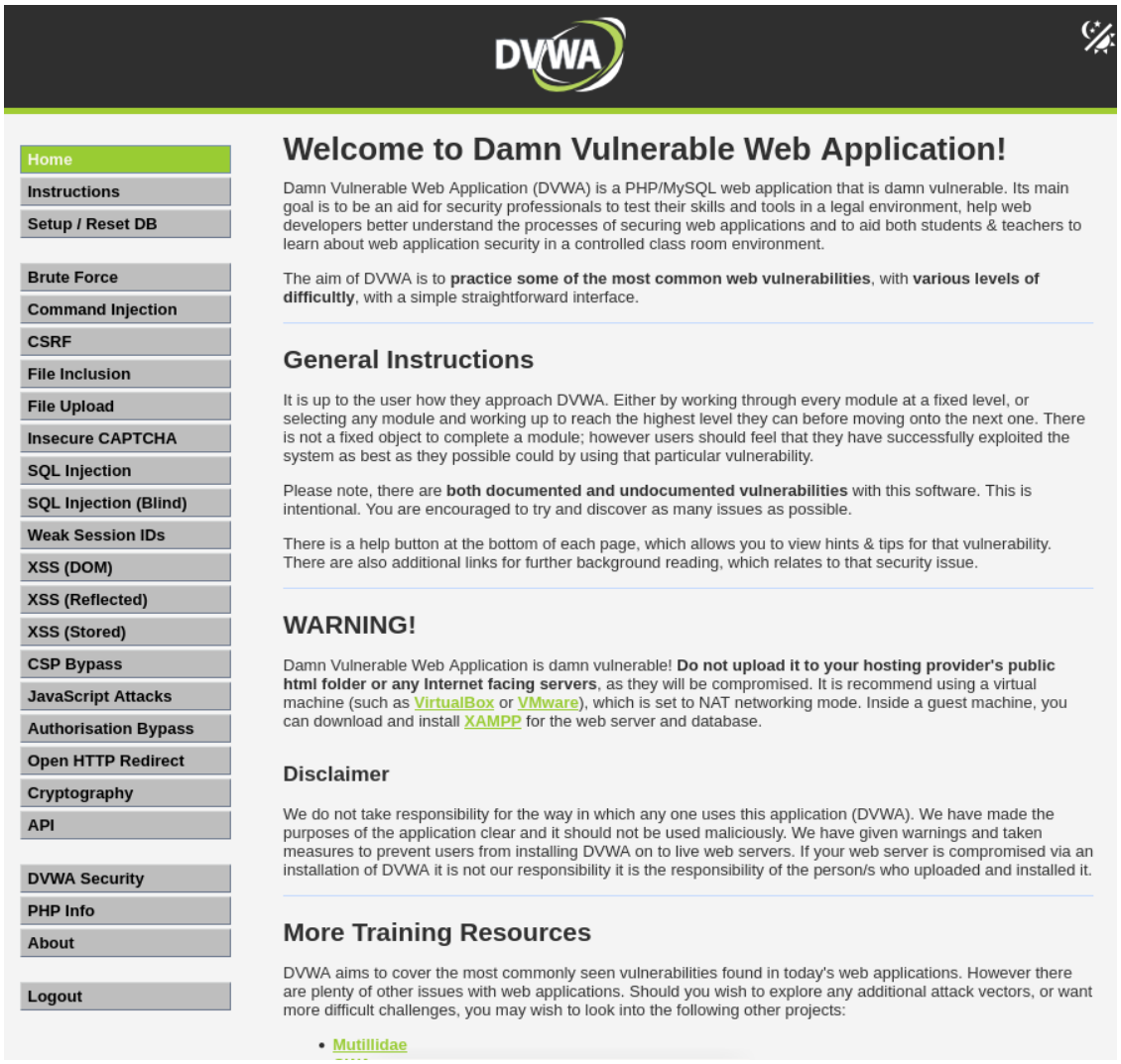


Ilustración 26 Acceso a DVWA

En el navegador, se accedió a **http://192.168.0.131/dvwa** y en la pantalla de login ingresamos las credenciales por defecto:

Usuario: admin

Contraseña: password

Después de pulsar **Login**, la aplicación validó las credenciales contra la base de datos y nos redirigió a la página de bienvenida de DVWA, lo que confirma que el acceso fue exitoso. La imagen corresponde a la pantalla principal que aparece tras iniciar sesión correctamente. Muestra el mensaje de bienvenida, el panel de navegación lateral con los distintos laboratorios (SQL Injection, XSS, File Inclusion, CSRF, etc.) y, normalmente, información introductoria sobre el propósito de DVWA y advertencias de uso. Ver esta pantalla significa que la base de datos y las tablas fueron creadas correctamente y que el sistema de autenticación está funcionando.

Configuración del módulo mod_rewrite y ajustes en apache2.conf para habilitar DVWA en Apache

Configuración de Apache para habilitar .htaccess mediante AllowOverride All

```
#
# LogLevel: Control the severity of messages logged to the error_log.
# Available values: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the log level for particular modules, e.g.
# "LogLevel info ssl:warn"
#
LogLevel warn

# Include module configuration:
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf

# Include list of ports to listen on
Include ports.conf


# Sets the default security model of the Apache2 HTTPD server. It does
# not allow access to the root filesystem outside of /usr/share and /var/www.
# The former is used by web applications packaged in Debian,
# the latter may be used for local directories served by the web server. If
# your system is serving content from a sub-directory in /srv you must allow
# access here, or in any related virtual host.
<Directory />
        Options FollowSymLinks
        AllowOverride None
        Require all denied
</Directory>

<Directory /usr/share>
        AllowOverride None
        Require all granted
</Directory>

<Directory /var/www/>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
</Directory>

#<Directory /srv/>
#       Options Indexes FollowSymLinks
```

Ilustración 27 Habilitación de htaccess mediante AllowOverride All

En el archivo de configuración principal de Apache (**/etc/apache2/apache2.conf**), por defecto la directiva **AllowOverride** suele estar establecida como **None** para el directorio raíz de la web (**/var/www/ o /var/www/html).** Esta configuración impide que los archivos .**htaccess** dentro de los directorios web puedan sobrescribir o definir reglas como las necesarias para el correcto funcionamiento de aplicaciones como DVWA. Modificar esta directiva a **AllowOverride All** permite que Apache lea y aplique las reglas contenidas en los archivos .**htaccess**, incluyendo reglas de **mod_rewrite** y control de acceso, esenciales para el enrutamiento y seguridad dentro de DVWA. Para hacer el cambio, se edita el archivo **apache2.conf** con un editor como nano (**sudo nano /etc/apache2/apache2.conf**), se busca la sección **<Directory /var/www/>** o correspondiente a la raíz web, y se cambia la línea:

AllowOverride None ===> AllowOverride All

Este cambio habilita las directivas internas en .**htaccess**. Finalmente, es importante reiniciar el servicio Apache con **sudo systemctl restart apache2** para que la nueva configuración tenga efecto.

Configuración del archivo .htaccess en DVWA



Ilustración 28 Ruta /var/www/html/dvwa/.htaccess

El archivo .**htaccess** ubicado dentro del directorio de DVWA contiene las reglas necesarias para controlar el comportamiento del servidor Apache y permitir el correcto funcionamiento de la aplicación. Antes de usarlo, se debe habilitar el módulo de reescritura con el comando **sudo a2enmod rewrite** y reiniciar el servicio con **sudo systemctl restart apache2**. Este archivo activa el motor de reescritura (**RewriteEngine On**) y establece condiciones para redirigir solicitudes que no correspondan a archivos o directorios reales hacia **index.php**, lo que permite manejar las rutas internas de DVWA. Además, se utiliza para verificar que el módulo **mod_rewrite** esté funcionando

correctamente y que las peticiones sean procesadas adecuadamente por la aplicación, garantizando que DVWA pueda gestionar sus páginas dinámicas y pruebas de seguridad sin errores de acceso.

- **RewriteEngine On:** Activa el motor de reescritura de URL de Apache, necesario para que las siguientes reglas funcionen.
- **RewriteBase /dvwa/:** Define la ruta base donde se aplicarán las reglas, en este caso el directorio /**dvwa**/. Esto le indica a Apache que todas las redirecciones y reglas se aplican dentro de esta carpeta específica.
- **RewriteCond %{REQUEST_FILENAME} !-f:** Indica que la regla siguiente solo se aplicará si la ruta solicitada **no corresponde a un archivo físico existente**. Por ejemplo, si el usuario solicita un archivo HTML o una imagen real, esta condición evita que se redireccione.
- **RewriteCond %{REQUEST_FILENAME} !-d:** Similar a la anterior, evita aplicar la regla si la ruta corresponde a un **directorio real** en el sistema.
- **RewriteRule ^(.*)$ index.php?q=$1 [L,QSA]:** Esta es la regla principal. Redirigir cualquier solicitud que no sea un archivo o carpeta existente hacia el archivo **index.php**, pasando la ruta solicitada como parámetro q.

RewriteEngine On / RewriteRule ^test_rewrite$ /index.php [L]: Esta parte sirve como prueba para verificar que el módulo **mod_rewrite** funciona correctamente. Si escribes en el navegador **http://192.168.0.131/dvwa/test_rewrite**, Apache debería redirigir automáticamente a **index.php**

Comprobación del funcionamiento de mod_rewrite en DVWA

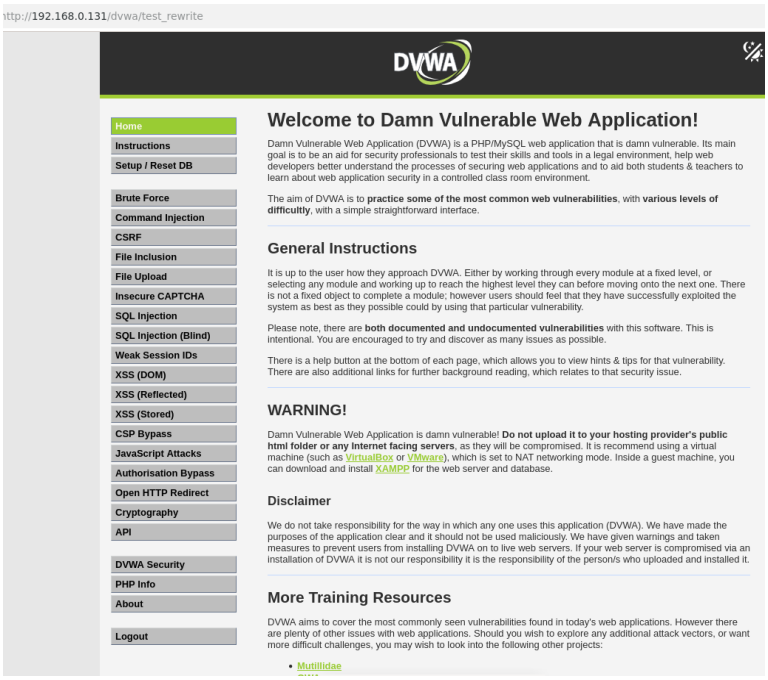


Ilustración 29 Navegador http://192.168.0.131/dvwa/test_rewrite

Para verificar que el módulo **mod_rewrite** está correctamente habilitado y que las reglas del archivo .**htaccess** funcionan, accedimos desde el navegador a la dirección: **http://192.168.0.131/dvwa/test_rewrite**. Al cargar esta URL, el servidor redirige automáticamente la solicitud hacia **index.php**, demostrando que el motor de reescritura de Apache está activo y procesando las reglas definidas en .**htaccess.** Esta prueba confirma que las rutas internas de DVWA se gestionan de forma correcta, lo que permite que la aplicación funcione sin errores de acceso ni problemas al cargar sus módulos. Esta verificación es importante, ya que sin el correcto funcionamiento de **mod_rewrite**, DVWA no podría redirigir las peticiones de sus diferentes laboratorios (como SQL Injection, XSS o File Inclusion), afectando el uso normal de la aplicación.

Escaneo de puertos y versiones de software con Kali Linux usando Nmap

Para este apartado se realizó un análisis detallado de vulnerabilidades a través de escaneo activo y se realizó un reporte de resultados obtenidos.

Verificación de interfaces red y la dirección IP del atacante

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:6e:13:6e brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.10/24 brd 192.168.200.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe6e:136e/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:a0:52:1a:61 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$
```

Ilustración 30 Interfaces red y dirección ip del atacante

Se realizó la verificación de interfaces de red y la dirección IP del atacante a través del comando ip a, el cual permitió evidenciar que la dirección del atacante es 192.168.200.10.

Identificación de hosts activos y puertos abierto con un Null scan o técnica sigilosa

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sN 192.168.200.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-28 20:47 EST
Nmap scan report for 192.168.200.1                                    faltó -O
Host is up (0.00019s latency).
All 1000 scanned ports on 192.168.200.1 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 52:55:C0:A8:C8:01 (Unknown)

Nmap scan report for 192.168.200.3
Host is up (0.00012s latency).
All 1000 scanned ports on 192.168.200.3 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:97:FE:2F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.200.8
Host is up (0.00056s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE         SERVICE
22/tcp open|filtered ssh
MAC Address: 08:00:27:94:4E:A0 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.200.10
Host is up (0.000018s latency).
All 1000 scanned ports on 192.168.200.10 are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 256 IP addresses (4 hosts up) scanned in 3.45 seconds

┌──(kali㉿kali)-[~]
└─$
```

Ilustración 31 Hosts activos y puertos abiertos

Una vez que se cuenta con la información del comando ip a, se procede con la ejecución del comando sudo nmap-sN 192.168.200.0/24, la ejecución de dicho comando permite

identificar los hosts activos en la red junto con los puertos abiertos a través de una técnica sigilosa o Null scan.

Una vez que se ejecutó el comando, se logra identificar que hay una dirección ip con el puerto 22 abierto y el servicio ssh activo, la dirección ip 192.168.200.8 representa la máquina objetivo.

Dirección IP de Debian

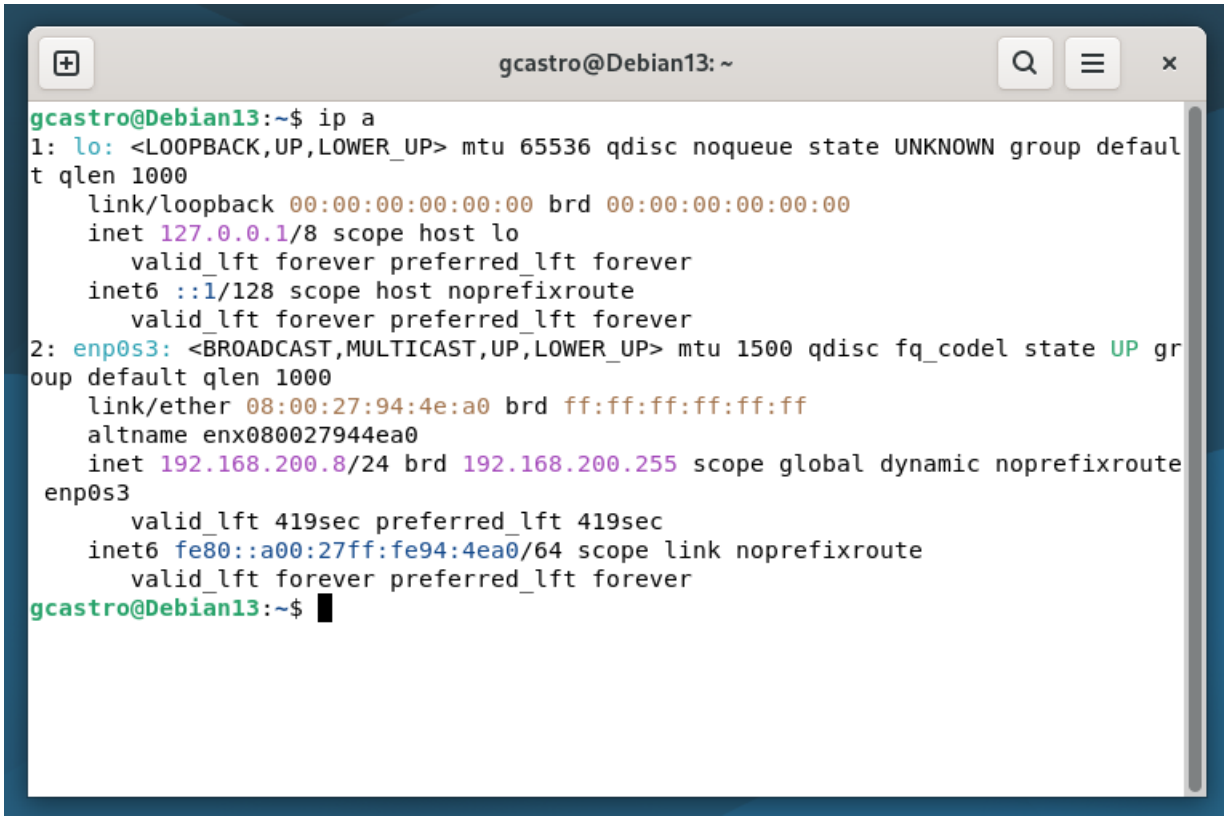


Ilustración 32 Dirección ip de la máquina objetivo

Para evidenciar que la dirección ip 192.168.200.8 es la dirección de la máquina objetivo, se accedió a Debian y a través de la ejecución del comando ip a, se logró determinar que la dirección comentada es efectivamente la dirección de la máquina objetivo.

Enumeración de versiones de servicios

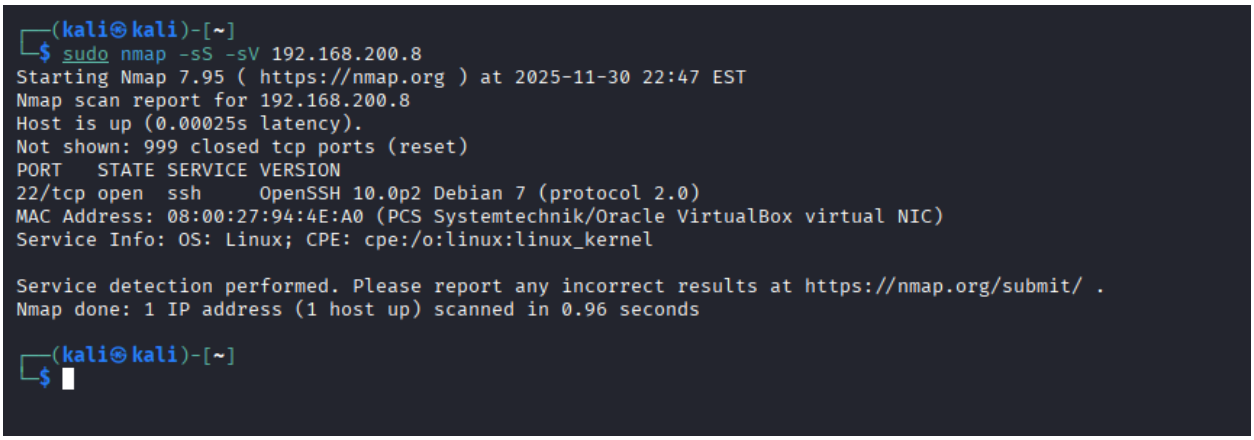


Ilustración 33 Versiones de los servicios activos

Para identificar las versiones de servicios y a su vez determinar posibles vulnerabilidades conocidas, se hizo la ejecución del comando sudo nmap -sS -sV 192.168.200.8, dicho comando permitió hacer una combinación de escaneo de puertos detección de servicios, por lo tanto, una vez ejecutado, se logra identificar que el host objetivo presenta el puerto 22/tcp abierto y que el mismo cuenta con la versión OpenSSH 10.0p2 Debian 7 con protocolo 2.0, una vez que se cuenta con dicha información, es relevante mencionar que la versión OpenSSH 10.0p2 en Debian 7 con protocolo 2.0 presenta vulnerabilidades críticas conocidas dentro de las cuales se destacan las presentes en el siguiente cuadro:

Vulnerabilidad	Riesgo / Implicaciones de seguridad
CVE-2024-6387 (regreSSHion)	Permite a un atacante remoto no autenticado ejecutar código con privilegios de root debido a una condición de carrera en el manejo de señales, lo cual convierte al puerto 22 en un vector importante de ataque si este no es actualizado o mitigado.
CVE-2024-6409 – Privilege Separation Child Process	Permite la explotación en el proceso hijo de separación de privilegios lo que genera la posible ejecución remota de código.

Protocol 2.0	El uso del protocolo 2.0 puede generar ataques de fuerza bruta si no se configuran los límites de intentos. Puede generar explotación de configuraciones débiles como los cifrados inseguros o los habilitados por defecto.

Verificación de la conectividad con el host objetivo

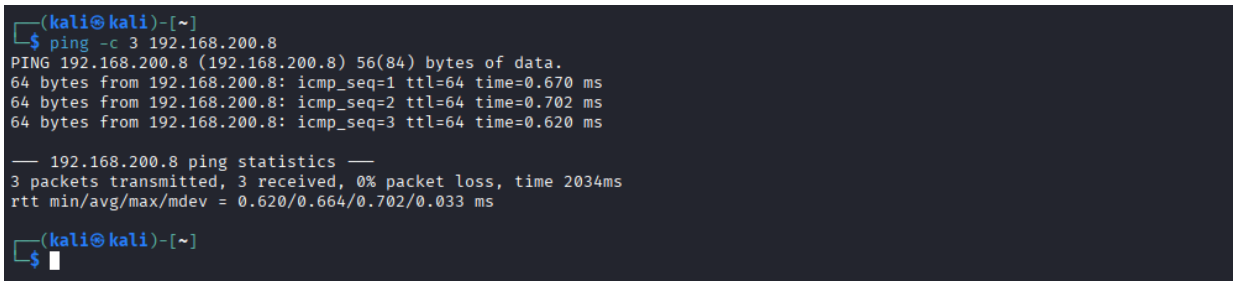


Ilustración 34 Ejecución de ping para verificar la conectividad con el host objetivo

Una vez se hizo la ejecución de los comandos para conocer la dirección ip del host objetivo, los puertos abiertos, entre otra información, se ejecutó el comando ping -c 3 192.168.200.8 para confirmar que el host responde a ICMP y medir la latencia, la imagen representa el envío de 3 paquetes y evidencia no se perdió ningún paquete, por lo tanto, el host es accesible y está listo para ser explotado.

Escaneo de versiones y puertos abiertos

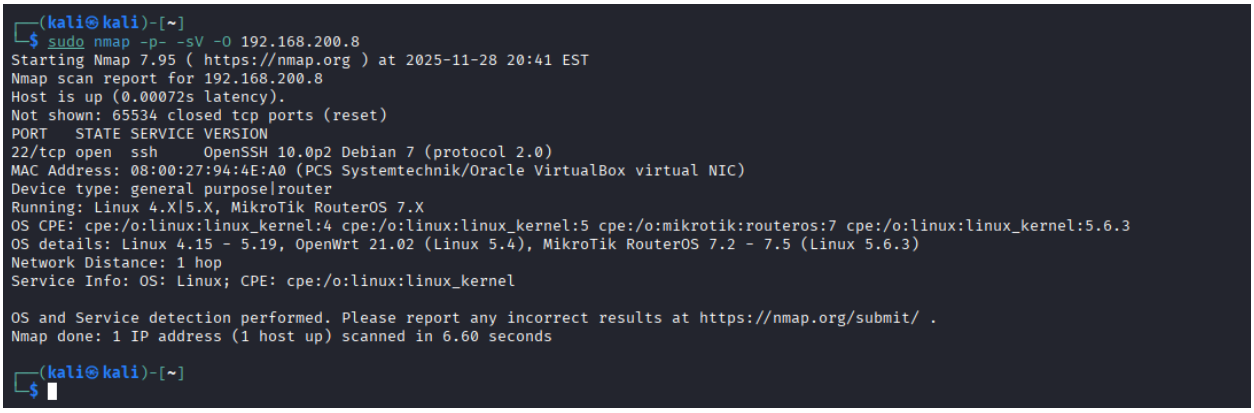


Ilustración 35 Comando sugerido / Versiones y puertos abiertos

Finalmente, se procedió a ejecutar el comando sudo nmap -p- -sV -O 192.168.200.8, el cual fue sugerido para realizar el escaneo de versiones y puertos abiertos, el comando evidencia el mismo resultado que el comando sudo nmap -sS -sV 192.168.200.8, el cual fue ejecutado anteriormente.

Simulación de ataque DoS desde Kali Linux contra servidor Apache

Ejecución y monitoreo del ataque DoS con evidencia visual del impacto en el servidor

En este sector se envió un ataque DDos con el comando **sudo ping -f** hacia la IP del Servidor Debian, el cual era **192.168.100.152** por lo tanto, se hizo uso del comando completo **sudo ping -f 192.168.100.152**. Este ataque se le conoce como ¨**ping flood**¨ envía pings de forma masiva y rápida hacia el servidor Debian. Como ya habíamos configurado una regla para bloquear ICMP desde la IP atacante (**192.168.100.148**), se esperaba que el servidor no respondiera a ese ataque evitando tanto tráfico.

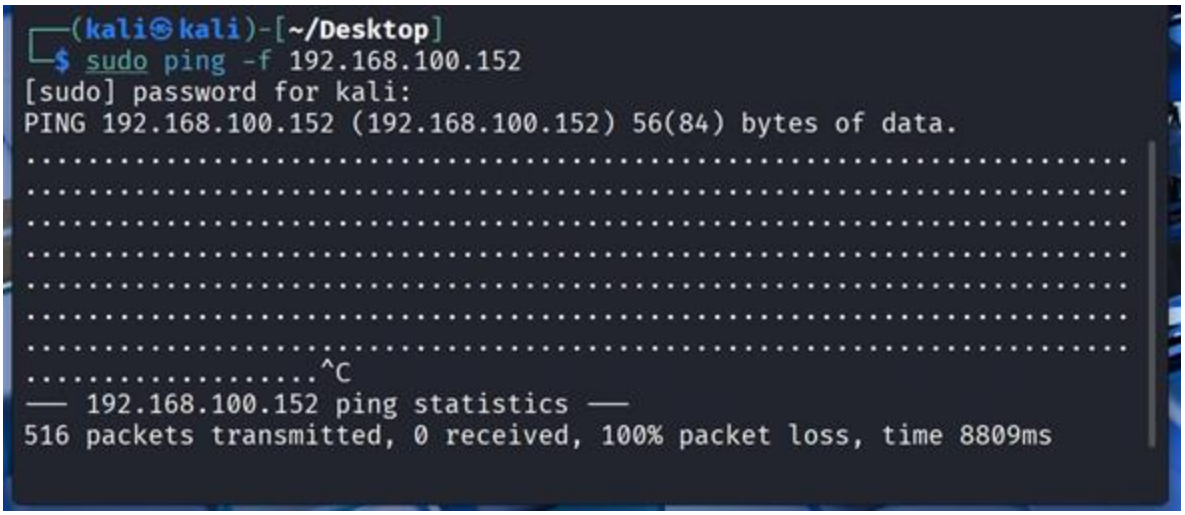


Ilustración 36 Ataque DDos con sudo ping -f

En la captura se observa que se enviaron **516 paquetes** y no se recibió ninguno, lo que confirma que el tráfico fue bloqueado exitosamente.

Y también se realizó un ataque **TCP (SYN flood al puerto 3389) esto con el objetivo de verificar si la regla** para bloquear paquetes del puerto **3389** desde Kali (**3ero**), estaba funcionando y efectivamente no recibió ni se documentó ningún paquete. Para este ataque se utilizó el comando: sudo hping3 -S -p 3389 -i u1000 192.168.100.152

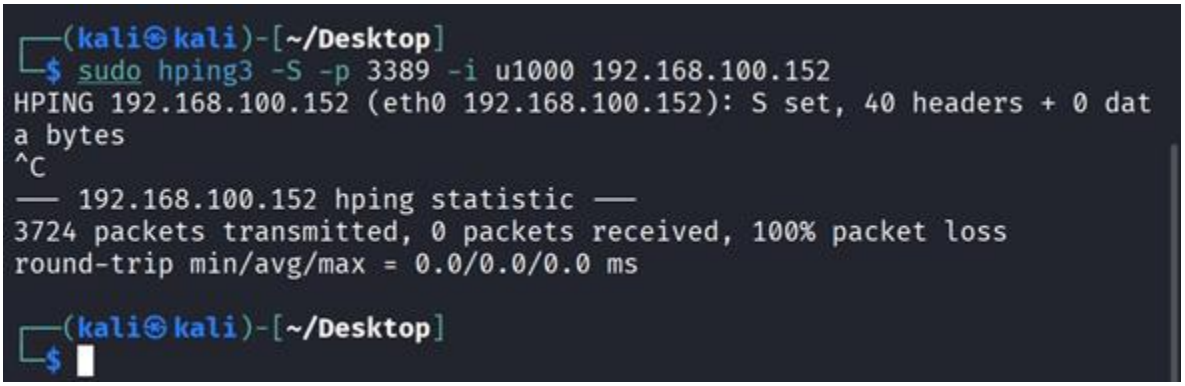


Ilustración 37 Ataque TCP

```
# Completed on Wed Nov 12 15:13:08 2025
debiancenfotec@debian-servidor:~$ sudo iptables -L -v
Chain INPUT (policy DROP 1310 packets, 337K bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 LOG        icmp --  any    any     192.168.100.152      anywhere             LOG level warn prefix "ICMP LOCAL: "
    0     0 ACCEPT     tcp  --  any    any     192.168.100.152      anywhere             tcp dpt:ms-wbt-server
  516 43344 DROP       icmp --  any    any     192.168.100.148      anywhere
 3725  149K DROP       tcp  --  any    any     192.168.100.148      anywhere             tcp dpt:ms-wbt-server

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy DROP 682 packets, 45425 bytes)
 pkts bytes target     prot opt in     out     source               destination
debiancenfotec@debian-servidor:~$
```

Ilustración 38 Listado de reglas de iptables

Configuración avanzada de reglas iptables en Debian para control de tráfico y seguridad

Implementación de políticas para registrar, permitir y bloquear paquetes ICMP y tráfico RDP con políticas DROP por defecto

En esta sección se explica cómo se configuraron las políticas de filtrado en el servidor Debian para protegerlo contra ataques **DoS**. Aunque el trabajo indicaba usar el comando ¨**service iptables save**¨. Este no está disponible en Debian, por lo que se optó por una solución alternativa válida y funcional, para obtener los permisos.

Primeramente, se instalaron las políticas por defecto y los comandos y reglas **DROP** para evitar tráfico no autorizado. Considerando que son tres reglas:

-P INPUT DROP

-P FORWARD DROP

-P OUTPUT DROP

```
debiancenfotec@debian-servidor: ~
Archivo  Editar  Ver  Buscar  Terminal  Ayuda
Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
debiancenfotec@debian-servidor:~$ sudo iptables -A INPUT -s 192.168.100.152 -p icmp -j LOG --log-prefix "I
CMP LOCAL: "
debiancenfotec@debian-servidor:~$ sudo iptables -A INPUT -s 192.168.100.152 -p tcp --dport 3389 -j ACCEPT
debiancenfotec@debian-servidor:~$ sudo iptables -A INPUT -s 192.168.100.148 -p icmp -j DROP
debiancenfotec@debian-servidor:~$ sudo iptables -A INPUT -s 192.168.100.148 -p tcp --dport 3389 -j DROP
debiancenfotec@debian-servidor:~$ sudo iptables -P INPUT DROP
debiancenfotec@debian-servidor:~$ sudo iptables -P FORWARD DROP
debiancenfotec@debian-servidor:~$ sudo iptables -P OUTPUT DROP
debiancenfotec@debian-servidor:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy DROP)
num  target     prot opt source               destination
1    LOG        icmp --  192.168.100.152      0.0.0.0/0            LOG flags 0 level 4 prefix "ICMP LOCAL:
 "
2    ACCEPT     tcp  --  192.168.100.152      0.0.0.0/0            tcp dpt:3389
3    DROP       icmp --  192.168.100.148      0.0.0.0/0
4    DROP       tcp  --  192.168.100.148      0.0.0.0/0            tcp dpt:3389

Chain FORWARD (policy DROP)
num  target     prot opt source               destination

Chain OUTPUT (policy DROP)
num  target     prot opt source               destination
debiancenfotec@debian-servidor:~$
```

Ilustración 39 Establecimiento de reglas en iptables

```
debiancenfotec@debian-servidor:~$ sudo bash -c "iptables-save > /etc/iptables/rules.v4"
debiancenfotec@debian-servidor:~$ cat /etc/iptables/rules.v4
# Generated by iptables-save v1.8.11 (nf_tables) on Wed Nov 12 15:13:08 2025
*filter
:INPUT DROP [1195:313353]
:FORWARD DROP [0:0]
:OUTPUT DROP [606:40614]
-A INPUT -s 192.168.100.152/32 -p icmp -j LOG --log-prefix "ICMP LOCAL: "
-A INPUT -s 192.168.100.152/32 -p tcp -m tcp --dport 3389 -j ACCEPT
-A INPUT -s 192.168.100.148/32 -p icmp -j DROP
-A INPUT -s 192.168.100.148/32 -p tcp -m tcp --dport 3389 -j DROP
COMMIT
# Completed on Wed Nov 12 15:13:08 2025
debiancenfotec@debian-servidor:~$
```

Ilustración 40 Establecimiento de reglas de filtrado de tráfico en iptables

Y después se procedió a colocar las demás reglas en específico como lo fueron:

- Registrar paquetes ICMP desde la IP local

sudo iptables -A INPUT -s 192.168.100.152 -p icmp -j LOG --log-prefix "ICMP LOCAL:

- Permitir tráfico TCP desde la IP local al puerto 3389

sudo iptables -A INPUT -s 192.168.100.152 -p tcp --dport 3389 -j ACCEPT

- Bloquear paquetes ICMP desde Kali

sudo iptables -A INPUT -s 192.168.100.148 -p icmp -j DROP

- Bloquear paquetes TCP al puerto 3389 desde Kali

sudo iptables -A INPUT -s 192.168.100.148 -p tcp --dport 3389 -j DROP

Las reglas mencionadas cumplen con los requisitos del trabajo: proteger el servidor contra tráfico malicioso desde Kali y permitir el tráfico legítimo desde la IP local. Cabe aclarar que como muestra la imagen anterior ¨ se utilizó el comando **iptables-save**¨ esto lo hace de manera manual guardando los comandos; en vez de usar **service iptables save**, que básicamente no permite Debian. Este comando guarda todas las reglas activas en el archivo **rules.V4** , que puede ser cargado automáticamente al reiniciar si se configura correctamente. Además, se utilizó el comando **cat** para ver dónde se guardaron todas esas reglas creadas para evitar y denegar el tráfico.

Luego, se verificó que las reglas estaban funcionando con el comando **sudo iptables -L**

-v que muestra los contadores de paquetes, y se observó que las reglas para **ICMP y TCP** desde Kali registraron actividad, lo que valida su efectividad.



Ilustración 41 Verificación de funcionalidad de reglas establecidas

Conclusiones

La realización del proyecto permitió profundizar en aspectos prácticos de seguridad en redes, especialmente en la configuración de firewalls, actualizaciones automáticas y protección contra ataques comunes. Ahora bien, es relevante recalcar que la combinación de IPTables y Fail2Ban se consideró como una estrategia efectiva para mitigar riesgos en servidores, y la implementación de DVWA fue útil para comprender vulnerabilidades reales y a su vez, identificar cómo mitigarlas.

Ahora bien, durante la ejecución del proyecto, se logró evidenciar la cantidad de incidentes de seguridad y riesgos presentes debido a no contar con controles o medidas que permitan generar una postura de seguridad robusta, ya que con la ejecución de unos cuantos comandos se logró conocer la dirección ip del objetivo y a partir de ello, realizar escaneos para identificar los puertos abiertos y versiones de servicios que al ser obsoletas o con poca protección, dan pase libre a atacantes para explotar las vulnerabilidades y de esta forma generar riesgos críticos. Esto no solo permite ampliar los conocimientos respecto a comandos que pueden utilizarse para realizar escaneos, detectar vulnerabilidades y mitigar riesgos, si no que también permite reflexionar sobre el cuidado que debe tenerse para evitar configuraciones débiles u obsoletas que afectan la seguridad y el rendimiento de los sistemas.

Además, la práctica realizada fue sumamente enriquecedora, ya que permitió explorar de forma controlada cómo se comporta un servidor Debian ante distintos tipos de ataques de denegación de servicio. Con esto se logró aplicar reglas de filtrado con IPTABLES para bloquear tráfico ICMP y TCP proveniente de la máquina atacante (Kali), y se utilizaron comandos como **ping -f** y hiping**3** para realizar los ataques tipo ping **flood** y **SYN flood.**

Durante el proceso, se aprendió a:

- Configurar políticas por defecto y reglas específicas para proteger el servidor.
- Verificar la efectividad de las reglas mediante contadores de paquetes (iptables -L -v).
- Guardar las reglas correctamente usando iptables-save en lugar de ,

adaptándose a las limitaciones de Debian.

- Utilizar herramientas de monitoreo como htop y iftop y el inspector de red de Firefox para observar el comportamiento del sistema en tiempo real.

Asimismo, el proyecto brindó la oportunidad de reforzar la importancia de aplicar medidas preventivas, validar configuraciones y documentar cada paso del proceso para garantizar la seguridad del entorno.La parte más interesante de la práctica fue descubrir una vulnerabilidad escalable y explotable, ya que aunque el sistema indicaba que no se estaban recibiendo paquetes durante el ataque **TCP flood**, el tráfico sí estaba llegando al servidor, como se evidenció en las capturas de red desde Firefox y herramientas de monitoreo.

Finalmente, los hallazgos demostraron que, en un entorno real, una mala configuración ya sea de **IPs** o **reglas de filtrado** pueden permitir que un ataque pase desapercibido, consumiendo recursos y comprometiendo el rendimiento del servidor o incluso si violase el **WAF** evidenciaría el enfrentamiento a incidentes críticos, lo cual refleja la importancia de la revisión profunda y el monitoreo activo, ya que los mismos son esenciales para garantizar una defensa efectiva.

Bibliografía

Cyosnet. (11 de junio de 2023). *www.youtube.com*. Obtenido de www.youtube.com: https://www.youtube.com/watch?v=wd7M1u9GnXI

debian. (s.f.). *security-tracker.debian.org*. Obtenido de security-tracker.debian.org: https://security-tracker.debian.org/tracker/CVE-2024-6387

íTecnoCode. (06 de octubre de 2025). *www.youtube.com*. Obtenido de www.youtube.com: https://www.youtube.com/watch?v=zD-0PR-L6i8

Kaplan, G. (01 de julio de 2024). *www.oligo.security*. Obtenido de www.oligo.security: https://www.oligo.security/blog/critical-openssh-vulnerability-cve-2024-6387-regresshion

Locos por Linux. (09 de agosto de 2023). *www.youtube.com*. Obtenido de www.youtube.com: https://www.youtube.com/watch?v=nmU_4PDit70

Redes Plus. (30 de junio de 2022). *www.youtube.com*. Obtenido de www.youtube.com: https://www.youtube.com/watch?v=PeMBfZ7ummg

Stevenson, V. (01 de agosto de 2023). *www.youtube.com*. Obtenido de www.youtube.com: https://www.youtube.com/watch?v=AtxRdhQuDyo

Integrantes del grupo: Génesis Castro Quesada, Jose Pablo Picado Chaves y Emmanuel Richmond Tasies.
Código del Curso: CIB-12 Aseguramiento de redes.

INSTRUMENTO DE EVALUACIÓN PARA TRABAJO						
Aspectos de formato	Puntaje – 0(NO) – 1(SÍ)					Recuerde incluir este instrumento de evaluación en su trabajo.
1. Realiza la portada de acuerdo con las indicaciones.			X			
2. Presenta el instrumento de evaluación.			X			
3. Sube el documento con formato pdf.			X			
4. Coloca el nombre al documento según lo especificado.			X			
5. Utiliza las fuentes de letra indicadas.			X			
6. Identifica claramente cada una de las partes solicitada en las instrucciones de actividades.			X			
7. No tiene errores ortográficos ni gramaticales.			X			
8. Enumera las páginas y presenta el trabajo según el orden establecido.			X			
9. Coloca la Bibliografía.			X			
10. Enumera y usa descripciones para las imágenes.			X			
Aspectos de contenido	1	2	3	4	5	
11. Fail2Ban filtra los ataques DoS					X	
12. DVWA está configurada corriendo con la BD configurada					X	
13. DVWA está con el archivo apache correctamente configurado.					X	
14. Escanea la máquina de Debian como se indica.				X		
15. Se muestra el ataque DoS antes de ser mitigado.						
16. Crear de forma correcta las reglas de ICMP.					X	
17. Crea las políticas generales de DROP.					X	
18. Guarda las configuraciones del servicio iptables					X	
19. Profundiza en el conocimiento adquirido del proyecto mediante 3 conclusiones.					X	

20. Entrega el Laboratorio a tiempo.					X
Puntaje Total: 100 pts 60pts	Puntos Obtenidos:		54 pts	Porcentaje Obtenido:	